

Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Davide Fissore^[0009–0002–5934–8776] and Enrico Tassi^[0000–0002–7783–528X]

Université Côte d’Azur, Inria, France
`davide.fissore@inria.fr` `enrico.tassi@inria.fr`

Abstract. We mechanize two operational semantics for PROLOG with cut and prove their equivalence in the Rocq prover.

The first semantics is closely related to the one introduced by Debray and Mishra [7] for studying the optimizations employed by PROLOG abstract machines [29,24]. This semantics is based on a stack of alternatives, where the cut operator is represented by a pointer to a shorter stack. The interpreter for the ELPI logic programming language is based on this semantics [9].

The second semantics is based on And-Or trees, in which the structure of alternative computations is made explicit and preserved throughout execution. This enables an explicit characterization of the cut operator as a function that prunes selected branches of the tree. The tree-based semantics allows us to formulate introduction and elimination rules for disjunction in the presence of cut, which, as expected, differ from those of classical logic.

As a case study, we use the tree-based semantics to prove the soundness of the determinacy analysis introduced in [12]. By exploiting the equivalence between the two semantics, we relate this result to the stack-based semantics of the ELPI interpreter.

1 Introduction

ELPI is a dialect of λ PROLOG (see [22,23,9,14]) used as an extension language for the ROCQ prover (formerly the COQ proof assistant). ELPI has become an important infrastructure component: several projects and libraries depend on it [15,3,18,10,11]. Other notable Rocq libraries based on ELPI include the Hierarchy-Builder library-structuring tool [6]; the Trocq proof-transfer tool [4,5] and Derive [27,28,13], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, ELPI is equipped with a static analysis for determinacy [12] to help users control backtracking. ROCQ users are familiar with functional programming, but not necessarily with logic programming; in particular, uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker allows users to specify which predicates should be considered deterministic and then verifies that these predicates behave like functions, i.e., that they commit to their first solution and leave no

choice points (places where backtracking could resume). A choice point corresponds to a suspended *alternative* in the execution trace. In the following, we use *choice point* and *alternative* as synonyms.

Intuitively, nondeterminism arises when a computation can be completed using two different rules. The static analysis in [12] excludes this situation in two steps. The first step, called *mutual exclusion*, checks that at most one rule can apply to a given query. This condition is satisfied if either (i) no two rules have overlapping heads, so that no query can unify with both, or (ii) the analysis accounts for the presence of cut in rule premises, so that once the cut is executed all remaining rules are discarded. The second step, called *rule checking*, verifies that each rule either (i) calls only functions, ensuring that no choice point remains after the premises of the chosen rule have been executed, or (ii) executes a cut after calling relations, in which case any choice points left by the call are discarded.

The static analysis introduced in [12] is supported by benchmarks that demonstrate its usefulness, but it is not accompanied by a proof of soundness. This paper reports our first steps toward mechanizing the soundness of this analysis. In particular, we focus on the control operator *cut*, which is pervasively used to write deterministic code and causes the semantics to depart from a purely logical reading.

We mechanize two operational semantics for PROLOG with cut. The first is a stack-based semantics that closely models ELPI’s implementation and is essentially the semantics given by Debray and Mishra [7, Sec. 4.3], which underlies standard PROLOG abstract machines [29,1,24]. This semantics admits efficient implementations, but it makes reasoning about the scope of cut difficult: alternatives are stored in a stack, in chronological order, and the cut operator is represented as a pointer to a shorter stack. The relation between the rule being cut and the affected stack frames is therefore not immediately visible.

To address this limitation, we introduce a semantics based on And-Or trees, in which the history of the computation is represented in a structured way: given a goal, each applicable rule generates an Or-branch, which is in turn followed by an And-branch for each premise. This enables an explicit characterization of the cut operator as a function that prunes selected branches of the tree. The tree-based semantics allows us to formulate introduction and elimination rules for disjunction in the presence of cut, which, as expected, differ from those of classical logic. For example, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut away by q_1 .

Finally, we use the tree-based semantics to prove the correctness of the static determinacy analysis from [12]. We prove in Rocq that any call to a predicate identified as deterministic leaves no additional Or-branches in the tree when it terminates. By composing this result with the equivalence between the two semantics, we obtain that any call to a deterministic predicate leaves no choice points on the stack used by the ELPI interpreter.

Our mechanization is axiom-free and is available at <https://github.com/FissoreD/elpi-formalization>.

```

func f int -> int.           % f is a deterministic predicate. The two rules
f 1 2.                       % have non-verlapping heads, i.e. 1 != 2
f 2 3.
pred r int -> int.          % r is a non-deterministic predicate. The heads
r 0 0.                       % overlap: the query for 0 has three applicable
r 0 1.                       % rules.
r 0 2 :- !.
func g int -> int.          % g is deterministic: if the first rule succeeds,
g A C :- r A B, f B C, !.    % the second one is cut away, as well as the
g D F :- f D E, f E F.      % choice points left by the call to r.

```

Fig. 1: Running example of a ELPI program

2 Preliminaries: syntax and informal semantics of cut

In the entire paper we use Figure 1 as our running example, and we start by describing how we represent a program like that one. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ -calculus style (no parentheses).

The description of a program is given by a pair (list $\mathbb{R} * \text{sigT}$) and is shared by both semantics. A program consists of a list of rules $\mathbb{R}$ together with a signature environment sig . We delay the discussion of signatures to section 6 where we describe the static analysis that concerns them.

A rule is a pair ($\text{Tm} * \text{list Atom}$), where the first element represents its head (a term) and a list of premises. Each premise is an Atom , i.e. either the cut operator or a predicate call. Terms Tm include predicate symbols, data symbols, variables, and term application. The concrete definition of these two algebraic types is given below:

```

Inductive Tm := Symb of S | Pred of P | Var of V | App of Tm & Tm.
Inductive Atom := cut | call of Tm.

```

The syntax tree allows variables to be applied and predicates to be mixed with data. These higher-order features play no role in the current paper that focuses on first order logic programming, but allows future extensions to full λ Prolog to be based on the same mechanization.

The set of variables V , predicates P and data symbols S are countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the `finmap` library [21]. The empty substitution is written ε , while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$.

The backchaining operation applies all rules to a query term; it is central to both semantics it has the following type:

$$\text{backchain} : (\mathbb{P} * \mathcal{F}_T * \text{Tm} * \Sigma) \rightarrow (\mathcal{F}_T * \text{list } (\Sigma * \text{list Atom}))$$

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the

$$\begin{array}{c}
\frac{}{\sigma(\text{Tm_D } d) = \text{Tm_D } d} \qquad \frac{}{\sigma(\text{Tm_P } p) = \text{Tm_P } p} \\
\\
\frac{\text{if } v \in \sigma \text{ then } r = \sigma[v] \text{ else } r = v}{\sigma(\text{Tm_V } v) = r} \qquad \frac{\sigma t_1 = t'_1 \quad \sigma t_2 = t'_2}{\sigma(\text{Tm_App } t_1 t_2) = \text{Tm_App } t'_1 t'_2}
\end{array}$$

Fig. 2: Rules for substitution application

set of variable names used so far ($\mathcal{F}_v$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

The function `unify` takes two terms t_1 and t_2 , together with a substitution σ , as input, and optionally returns a substitution σ' that extends σ . Therefore, if t_1 and t_2 unify under σ , then $\sigma' t_1 = \sigma' t_2$, where σt denotes substitution application (see fig. 2) and $=$ denotes syntactic equality.

In order to be axiom-free, our mechanization implements a unification algorithm that we shortly describe in section 6. The two semantics and the static analysis use the very same unification algorithm, hence simply assuming the existence of one would have sufficed for the equivalence proofs. The case study in section 6 also requires a few standard properties of this algorithm, like correctness and completeness.

2.1 The cut operator

There are quite a few variants of the cut operator in the literature. The one we are interested in, i.e. the one used in ELPI, is known as *hard cut* (see, e.g., the documentation of SWI-PROLOG [26]). Whenever the `backchain` function returns a non-empty list of alternatives, the first one is explored immediately, while the others are suspended as choice points. Within a given alternative, premises are processed from left to right, executing each atom in turn. When a cut is encountered, it discards (i) all choice points created by `backchain` for the current call and (ii) all choice points created while executing the premises to the left of the cut.

For example, consider the program in fig. 1 and the query `g 0 R`. Both rules for `g` apply to this query. Backchaining therefore returns the following two alternatives:

$$[(\sigma_1, [\text{r A B}, \text{f B C}, !]), (\sigma_2, [\text{f D E}, \text{f E F}])]$$

with $\sigma_1 := \{\text{A} \mapsto 0, \text{C} \mapsto \text{R}\}$ and $\sigma_2 := \{\text{D} \mapsto 0, \text{F} \mapsto \text{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_v$ any further.

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1 , i.e. `r 0 B`. The remaining alternatives are stored as choice points.

Backchaining $\mathbf{r} \ 0 \ \mathbf{B}$ returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [!])]$ where $\sigma_3 := \sigma_1 + \{\mathbf{B} \mapsto 0\}$; $\sigma_4 := \sigma_1 + \{\mathbf{B} \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{\mathbf{B} \mapsto 2\}$.

The next step of interpretation continues with $\mathbf{f} \ \mathbf{B} \ \mathbf{C}$ under σ_3 , that is, $\mathbf{f} \ 0 \ \mathbf{R}$, and leaves $[(\sigma_4, []), (\sigma_5, [!])]$ as new choice points. This time, backchaining fails (no rule for $\mathbf{f}$ matches input 0). We therefore backtrack to the most recently introduced choice point and retry $\mathbf{f} \ \mathbf{B} \ \mathbf{C}$ under σ_4 , i.e. $\mathbf{f} \ 1 \ \mathbf{R}$. This succeeds with $\sigma_6 := \sigma_4 + \{\mathbf{R} \mapsto 2\}$.

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for $\mathbf{r}$ and one from the second rule for $\mathbf{g}$ (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for $\mathbf{g}$ was selected. In contrast, a cut does not discard choice points created earlier; in this example, there are none. The final solution is: $\{\mathbf{A} \mapsto 0, \mathbf{B} \mapsto 1, \mathbf{C} \mapsto \mathbf{R}, \mathbf{R} \mapsto 2\}$.

In sections 3 and 4, we provide fully mechanized operational semantics for PROLOG with cut. They differ in how they represent the ongoing computation, in particular how alternatives are stored and how they are pruned by cut.

Notational conventions In the following section we note $\mathbb{B}$ for the boolean type, and $\top$ and $\perp$ for **true** and **false**. The constructors of the option type are written $\cdot t$ for **Some** t and $\square$ for **None**. Following the Mathematical Components library, on which we depend, we use $\mathbf{x}.1$ and $\mathbf{x}.2$ to denote the first and second projection applied to the pair $\mathbf{x}$. List concatenation is the infix $++$, while list comprehension (i.e. mapping a function on a list) is written $[\mathbf{seq} \ \mathbf{f} \ \mathbf{x} \mid \mathbf{x} <- \mathbf{l}]$. The function `odflt` $\mathbf{d} \circ$ extracts the value from the option $\mathbf{o}$ and returns the default $\mathbf{d}$ if $\mathbf{o}$ is $\square$.

3 And-Or Tree semantics

An And-Or tree is a stratified, variable-width tree, defined in Rocq as in fig. 3. It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer.

```

Inductive tree :=
| KO | OK                               (* failure and success *)
| Todo of Atom                          (* unexplored branch *)
| Or   of option tree &  $\Sigma$  & tree      (*  $A \vee B_\sigma$  *)
| And  of tree & list Atom & tree.      (*  $A \wedge_{B_0} B$  *)

```

Fig. 3: Definition of And-Or tree

The tree is built incrementally. The **Todo** node represents an unexplored branch (an **Atom**). When the atom is a **call**, we use the backchaining procedure from section 2 to compute the two layers to attach to the tree: alternatives form the **Or**-layer, and the premises of each applicable rule form the corresponding **And**-layer. This expansion step is performed by the **step** procedure described in section 3.2.

In addition to the **Todo** node we have two distinguished nodes, **KO** and **OK**, which represent respectively the smallest failed and successful tree.

The **Or** node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A . The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ , which becomes the current substitution when backtracking to B .

The **And** node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to the first choice point in A . We call B_0 the *reset point* for B : it represents the continuation for all alternatives of A except the first. That is, when backtracking occurs within A , the subtree B is replaced by the atoms in B_0 . An example is shown in section 3.3.

A graphical representation of a tree is shown in fig. 5b. For compactness, we depict **And** and **Or** nodes as n -ary, with children associating to the right. The **KO** node acts as the terminating element of an **Or** list, while **OK** is the terminating element of an **And** list.

3.1 The semantics

The tree is constructed incrementally by two recursive functions, **step** and **prune**. Both traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved thanks to **next_tree** (in fig. 4). When this node is **OK** we say the entire tree is a *success*, when it is **KO** we say the tree *failed*, otherwise the tree is *incomplete*. In fig. 5 we mark with a black arrow the result of **next_tree**. The type of these two functions is given below.

$$\text{step} : (\mathbb{P} * \mathcal{F}_v * \Sigma * \text{tree}) \rightarrow (\mathcal{F}_v * \text{tag} * \text{tree})$$

$$\text{prune} : (\mathbb{B} * \text{tree}) \rightarrow \text{tree} + \square$$

The **step** routine, among its outputs, also returns a **tag**. We delay the definition in the very following section. **test**

Since all functions must terminate in Rocq, we define the transitive closure of **step** and **prune** as the inductive relation **runT**. More precisely **runT** iterates calls to **step** on incomplete trees to expand **Todo** nodes. When a tree fails it calls **prune** to find the next alternative and continues from there, if one exists.

3.2 The step procedure

The **step** procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a **tag**, and an updated tree.

The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

The `tag` is implemented as follows:

```
Inductive tag := Expanded | CutBrothers | Failed | Success.
```

It indicates which kind of step was performed. `Failure` and `Success` are returned for trees that satisfy, respectively, the `failed` and `success` tests. `CutBrothers` indicates the interpretation of a superficial cut: `step` was called on an And-layer and its `next_tree` is the cut atom. Finally, `Expanded` accounts for the interpretation of predicate calls and non-superficial cuts.

The `step` procedure evaluates atoms. In particular, it leaves the tree unchanged when the tree is *success* or *failed*.

Lemma 1 (succ_step_iff). $\forall \mathcal{P} \, v \, \sigma \, t, \text{success } t \leftrightarrow \text{step}(\mathcal{P}, v, \sigma, t) = (v, \text{Success}, t)$

Lemma 2 (fail_step_iff). $\forall \mathcal{P} \, v \, \sigma \, t, \text{failed } t \leftrightarrow \text{step}(\mathcal{P}, v, \sigma, t) = (v, \text{Failed}, t)$

The `step` procedure expands `Todo` nodes by calling `backchain`, which returns a list l of alternatives. If l is empty, then the current call atom is replaced by `KO`. Otherwise, it is replaced by a right-skewed tree of `Or` nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is `OK`; otherwise, it is a right-skewed tree of `And` nodes.

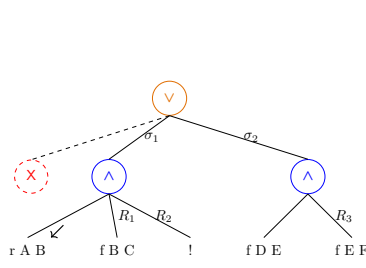
Figure 5 depicts the tree corresponding to the running example (section 2.1) as it undergoes calls to `step` and `prune`. We run query `q 0 R` against the program P in fig. 1. Let c_0 , c_1 , and c_2 be the children of the root. By construction, c_0 is the $\square$ tree, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P . Note that c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in section 2.1. Reset points are defined as follows: $R_1 := [\text{f } Y \, Z, !]$, $R_2 := [!]$, and $R_3 := \text{f } Y \, Z$. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of `step` performing a tree expansion via backchain appear in figs. 5c to 5e.

```
Fixpoint next σ t : Σ * tree := Definition next_subst σ t := (next σ t).1.
  match t with                    Definition next_tree t := (next ε t).2.
  | Todo _ | KO | OK => (σ, t)
  | Or □ σ' B => next σ' B        Definition success t := next_tree t == OK.
  | Or ·A _ => next σ A           Definition failed t := next_tree t == KO.
  | And A _ B =>                 Definition incomplete t :=
    let (σ', pA) := next σ A in   if next_tree t is Todo _ then ⊤ else ⊥.
    if pA == OK then next σ' B
    else (σ', pA)
  end.
```

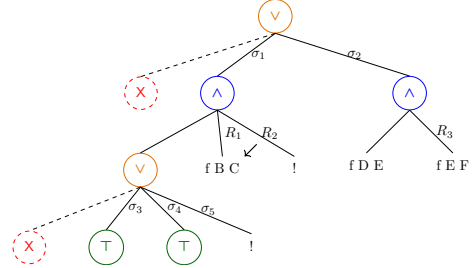
Fig. 4: `next` and derived functions

$g \ 0 \ R$

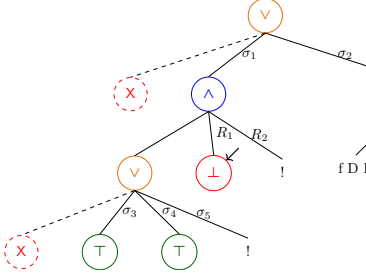
(a) Initial query



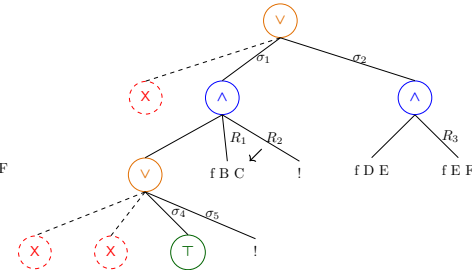
(b) Tree after **step** on $g \ 0 \ R$



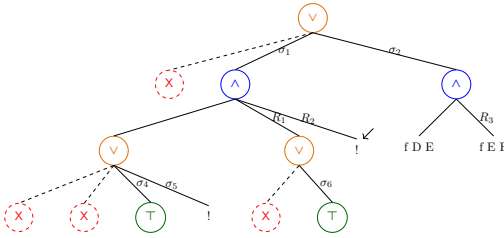
(c) **step** on fig. 5b



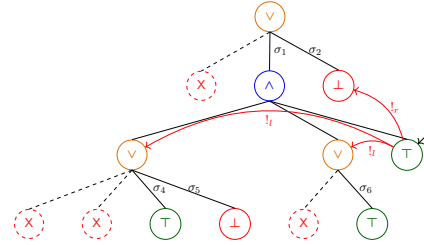
(d.1) **step** on fig. 5c



(d.2) **prune** on fig. 5d.1



(e) **step** on fig. 5d.2



(f) **step** on fig. 5e

Fig. 5: Execution in the tree semantics. The substitutions $\sigma_1 \dots \sigma_6$ are from section 2.1. R_1 , R_2 , and R_3 are reset points and correspond respectively to the lists of atoms $[f \ Y \ Z, \ !]$, $[!]$, and $[f \ Y \ Z]$.

The interpretation of a cut The step corresponding to a cut performs two actions: (i) the cut node is replaced by OK; and (ii) the subtrees in the scope of Cut are pruned. More precisely, (ii) prunes the left siblings of the Cut node (in the same And-layer, denoted $!_l$) and prunes the right uncles of Cut (in the one-level-up Or-layer, denoted $!_r$).

An example of the impact of cut is shown in fig. 5f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. Note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by K0. This amounts to discarding the third rule of predicate $\mathbf{r}$.

3.3 The prune procedure

When **backchain** returns an empty list, **step** produces a tree that *failed* and the computation must backtrack. The **prune** procedure is responsible for producing the new tree from which the computation can continue.

In fig. 5d.1 we encounter a failure because no rule applies to the atom $\mathbf{f} \ \mathbf{B} \ \mathbf{C}$ under substitution σ_3 , which maps $\mathbf{B}$ to 0. The **prune** procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the OK node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the K0 node with the information stored in the reset point R_1 . This restores the call $\mathbf{f} \ \mathbf{B} \ \mathbf{C}$, which can then be reconsidered under substitution σ_4 .

Furthermore, **prune** serves another important purpose. Its behavior depends on the boolean flag it receives as input: **prune** $\perp t$ recursively removes all failures (if any) from t , whereas **prune** $\top t$ removes the first success in t . For example, the tree in fig. 5f contains a successful path that maps $\mathbf{R}$ to 2. Calling **prune** on this tree returns $\square$, since there are no alternatives in the tree after the success.

Some interesting properties of **prune** are shown below; they illustrate how **prune** complements **step** with respect to failed, successful, and incomplete trees.

Lemma 3 (incomplete_prune_id). $\forall b \ t, \text{incomplete } t \rightarrow \text{prune } b \ t = t$

Lemma 4 (prune_Some). $\forall b \ t \ t', \text{prune } b \ t = t \rightarrow \text{failed } t' = \perp$

Lemma 5 (prune_None). $\forall t \ t', \text{prune } \perp t = \perp \rightarrow \text{failed } t' = \top$

3.4 The runT relation and its properties

The inductive relation **runT** has the following type:

$$\text{runT} : \mathbb{P} \times \mathcal{F}_L \times \Sigma \times \mathbb{T} \rightarrow ((\Sigma, \mathbb{T} + \square) + \square, \mathbb{B}, \mathcal{F}_L)$$

It takes a program, a set of variables, an initial substitution σ , and a tree t . The outputs are an optional result r , a boolean b , and an updated set of variable names.

The main result r is $\square$ when the execution fails; otherwise when $r = \cdot(\sigma', t')$ the execution succeeded and produced the solution σ' ; t' represents the remaining, not-yet-explored alternatives. The boolean b records whether a superficial cut was evaluated during execution.

The **runT** predicate has four cases, depicted as inference rules in fig. 6. (*StopT*) If **next_tree** t is a success, then the substitution σ' is extracted from t (starting from σ) and the input tree is cleaned of its successful path. (*StepT*) If **next_tree** t is an atom, then **step** is invoked to evaluate t , and **runT** is called recursively on the updated tree. (*BackT*) If **next_tree** t is a failure, then **prune** is called to clear the failed path; if the resulting cleaned tree exists, **runT** is called recursively on it. Finally, (*FailT*) accounts for trees with no solutions.

$$\begin{array}{c}
\frac{\text{success } t \quad \text{next_subst } \sigma \ t \rightsquigarrow \sigma' \quad \text{prune } \top \ t \rightsquigarrow t'}{\text{runT } \mathcal{P} \ v \ \sigma \ t \rightsquigarrow (\cdot(\sigma', t'), \perp, v)} \text{StopT} \\
\\
\frac{\text{incomplete } t \quad \text{step } \mathcal{P} \ v \ \sigma \ t \rightsquigarrow (v', tg, t') \quad \text{runT } \mathcal{P} \ v' \ \sigma \ t' \rightsquigarrow (r, b, v'')}{\text{runT } \mathcal{P} \ v \ \sigma \ t \rightsquigarrow (r, (tg = \text{CutBrothers}) \vee b, v'')} \text{StepT} \\
\\
\frac{\text{failed } t \quad \text{prune } \perp \ t \rightsquigarrow \cdot t' \quad \text{runT } \mathcal{P} \ v \ \sigma \ t' \rightsquigarrow r}{\text{runT } \mathcal{P} \ v \ \sigma \ t \rightsquigarrow r} \text{BackT} \\
\\
\frac{\text{prune } \perp \ t \rightsquigarrow \square}{\text{runT } \mathcal{P} \ v \ \sigma \ t \rightsquigarrow (\square, \perp, v)} \text{FailT}
\end{array}$$

Fig. 6: Rule system for **runT**

The set of rules defining **runT** is deterministic.

Lemma 6 (runT-det). $\forall \mathbf{p} \ v_0 \ \sigma \ t \ r \ r' \ b \ b' \ v \ v', \text{runT } \mathbf{p} \ v_0 \ \sigma \ t \ r \ b \ v \rightarrow$

$$\text{runT } \mathbf{p} \ v_0 \ \sigma \ t \ r' \ b' \ v' \rightarrow r' = r \wedge v' = v \wedge b' = b.$$

The proof is by induction on the first **runT** derivation and is immediate, because the rules are selected based on **next_tree** t . In particular, the hypotheses **success** t , **incomplete** t , and **failed** t are mutually exclusive. Moreover, by lemma 5, the hypothesis of *FailT* implies that t failed; hence *FailT* is exclusive with *StopT* and *StepT*, and it is also exclusive with *BackT* since they impose different requirements on the output of **prune**.

The boolean output of **runT** allows us to state interesting properties about the **Or** node in the presence of cut. Here we report only a selection of those we proved.

Lemma 7 (runSST-or). $\forall \mathbf{p} \ v \ v' \ \sigma \ \sigma' \ A \ A' \ \sigma_1 \ B, \text{runT } \mathbf{p} \ v \ \sigma \ A \ \cdot(\sigma', \cdot A') \top v' \rightarrow$

$$\text{runT } \mathbf{p} \ v \ \sigma \ (\cdot A \vee B_{\sigma_1}) \ \cdot(\sigma', \cdot(\cdot A' \vee KO_{\sigma_1})) \perp v'.$$

Lemma 8 (runNT-or). $\forall \mathbf{p} \ v \ v' \ \sigma \ A \ \sigma_1 \ B, \text{runT } \mathbf{p} \ v \ \sigma \ A \ \square \top v' \rightarrow$

$$\text{runT } \mathbf{p} \ v \ \sigma \ (\cdot A \vee B_{\sigma_1}) \ \square \perp v'.$$

Lemma 9 (runNF_or). $\forall p \ v_0 \ v_1 \ v_2 \ \sigma \ A \ \sigma_1 \ \sigma_2 \ B \ B' \ b,$

$$\begin{aligned} & \text{runT } p \ v_0 \ \sigma \ A \ \square \perp v_1 \rightarrow \text{runT } p \ v_1 \ \sigma_1 \ B \cdot (\sigma_2, B') \ b \ v_2 \rightarrow \\ & \quad \text{let } sR := \text{omap } (\text{fun } x \Rightarrow \square \vee x_{\sigma_1}) \ B' \ \text{in} \\ & \text{runT } p \ v_0 \ \sigma \ (\cdot A \vee B_{\sigma_1}) \cdot (\sigma_2, sR) \perp v_2. \end{aligned}$$

Lemmas 7 to 9 correspond to the introduction rules for disjunction. If A executes a cut, one *cannot* obtain $A \vee B$ from B : the execution of A , whether it ultimately succeeds or fails, makes the success of B irrelevant (the cut discards it). In the first lemma, B is forced to be $K0$; in the second one, the failure of A absorbs B . The last lemma is connected to the depth-first exploration of the tree: proving a disjunction from its right-hand side can only occur after the left-hand side has been disproved. Depicted as rules:

$$\frac{A \quad (A \text{ cuts})}{A \vee B} (\vee_{il}) \quad \frac{\neg A \quad (A \text{ cuts})}{\neg(A \vee B)} (\vee_{ir1}) \quad \frac{\neg A \quad B \quad (A \text{ does not cut})}{A \vee B} (\vee_{ir2})$$

Lemmas 10 and 11 are elimination rules for disjunction.

Lemma 10 (run_orSST). $\forall p \ v \ v' \ \sigma \ \sigma' \ \sigma_1 \ A \ A' \ B \ B',$

$$\begin{aligned} & \text{runT } p \ v \ \sigma \ (\cdot A \vee B_{\sigma_1}) \cdot (\sigma', \cdot (\cdot A' \vee B'_{\sigma_1})) \perp v' \rightarrow \\ & \quad \exists b, \text{runT } p \ v \ \sigma \ A \cdot (\sigma', \cdot A') \ b \ v' \wedge B' = \text{if } b \text{ then } K0 \text{ else } B. \end{aligned}$$

Lemma 11 (run_orSNT). $\forall p \ v \ v' \ \sigma \ \sigma' \ \sigma_1 \ A \ B \ B',$

$$\begin{aligned} & \text{runT } p \ v \ \sigma \ (\cdot A \vee B_{\sigma_1}) \cdot (\sigma', \cdot (\square \vee B'_{\sigma_1})) \perp v' \rightarrow \\ & \quad (\exists b, \text{runT } p \ v \ \sigma \ A \cdot (\sigma', \square) \ b \ v' \wedge \text{if } b \text{ then } B' = K0 \text{ else } \text{prune } \perp B = \cdot B') \vee \\ & \quad (\exists v_2 \ b, \text{runT } p \ v \ \sigma \ A \ \square \perp v_2 \wedge \text{runT } p \ v_2 \ \sigma_1 \ B \cdot (\sigma', \cdot B') \ b \ v'). \end{aligned}$$

From $A \vee B$, when A is not inert (i.e. not already explored), we cannot deduce A or B independently. Instead, we can only derive a weakened conclusion of the form $\top \vee B'$, where B' provides no information in the case where the exploration of A performs a cut. This yields an elimination rule that is stronger than the usual one (depicted on the left). If A does not cut, the elimination rule has the usual two premises, together with the additional constraint that whenever B holds, A must have failed. This again reflects the depth-first traversal of the proof tree.

$$\frac{A \vee B \quad \frac{[A] \quad C}{C} (A \text{ cuts})}{C} (\vee_{e1}) \quad \frac{A \vee B \quad \frac{[A] \quad C}{C} \quad \frac{[\neg A, B] \quad C}{C} (A \text{ does not cut})}{C} (\vee_{e2})$$

It is worth recalling that the “ A cuts” side-condition in the rules above is precisely the boolean result returned by **runT**. Without this information, it is impossible to formulate these rules: it is not merely a matter of whether A contains a syntactic occurrence of cut, but whether that cut is actually executed during the (dis)proof of A (in rules $(\vee_{ir1})$ and $(\vee_{ir2})$). Indeed, an atom preceding the cut may fail, so the cut is never reached.

As anticipated in the introduction, these lemmas contradict the commutativity of disjunction.

4 Stack-based semantics

The stack-based semantics we present is very close to what is implemented by the ELP1 interpreter. This semantics is similar to the one proposed by Pusch in [24], which is in turn closely related to the semantics given by Debray and Mishra in [7, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [29,1].

The execution state is a stack of alternatives. Each alternative is a list of goals. Intuitively it amounts to a disjunctive normal form: each stack item is a self contained computation, coupling a substitution with all the goals to be solved.

Goals pair an atom with a list of the *cut-to* alternatives, making the two datatypes mutually recursive, but these alternatives have a very different role. They have to be understood as pointers to lower levels of the stack itself. This datum is used to implement the cut, i.e. to prune the stack of alternatives.

Inductive `alts` := `nilA` | `consA of` (Σ * `goals`) & `alts`
with `goals` := `nilG` | `consG of` (`Atom` * `alts`) & `goals`

$$\mathcal{B}_S(p, v, t, \sigma, a, g) = \left[\sigma', ([(p, q, a) \mid q \in b] ++ g) \mid (\sigma', b) \in \mathbf{bc}(p, v, t, \sigma) \right].$$

$$\frac{}{\text{runS } \mathcal{P} \ v \ ((\sigma, \emptyset) :: a) \rightsquigarrow \cdot(\sigma, a)} \text{StopS} \quad \frac{\text{runS } \mathcal{P} \ v((\sigma, g) :: a) \rightsquigarrow r}{\text{runS } \mathcal{P} \ v((\sigma, (\text{cut}, a) :: g) :: _) \rightsquigarrow r} \text{CutS}$$

$$\frac{\mathcal{B}_S(\mathcal{P}, v, t, \sigma, a, g) = (v_1, a') \quad \text{runS } \mathcal{P} \ v_1 \ (a' ++ a) \rightsquigarrow r}{\text{runS } \mathcal{P} \ v \ ((\sigma, (\text{call } t, _) :: g) :: a) \rightsquigarrow r} \text{CallS} \frac{}{\text{runS } \mathcal{P} \ _ \rightsquigarrow \square} \text{FailS}$$

Fig. 7: Rule system for `runS`

The stack-based semantics is described by the `runS` inductive predicates.

$$\text{runS} : \mathcal{T} \times \text{alts} \rightarrow (\Sigma \times \text{alts}) + \square$$

The `runS` predicates relates a set of variables and a list of alternatives to optional result. $\square$ stands for failure, i.e., no solution, whereas $\cdot(\sigma, a)$ represent success with σ begin the solution and a as the list of non-yet-explored alternatives.

The rule system for `runS` is given in fig. 7 and is made by 4 cases. We distinguish two main cases. If we run out of alternatives we fail: (*FailS*) stops the execution and return $\square$. If the list of alternatives is $(\sigma_0, h) :: a$, then we look at the list of goals h (under the substitution σ_0). If h is empty, (*StopS*) stops the execution with a success σ_0 leaving a as the unexplored alternatives. If h is not empty we look at its head goal (t, ca) where t is the atom and ca is its cut-to alternatives. If t is a cut rule (*CutS*) continues to evaluate the tail of h under with alternatives ca . Finally, when t is a call, rule (*CallS*) backchains: for each rule it pairs each premise with the current stack of alternatives, and prepends the obtained goals to the tail of h . Note that if backchain returns the empty list, (*CallS*) second premise amounts to exploring the next item in the current stack.

4.1 Example of execution

We propose an example of the execution of `runS` in fig. 8. The arrows in the images represent the cut-to pointers.

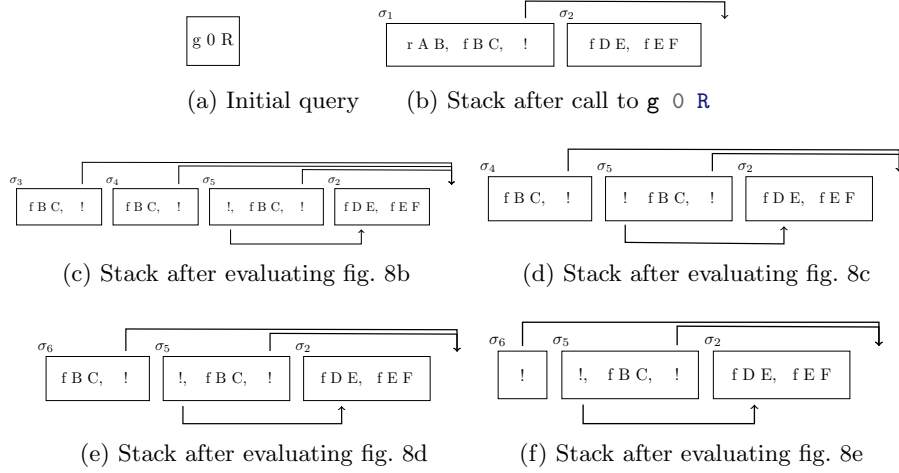


Fig. 8: Example of execution

We have not drawn arrows coming out of call atoms since the interpreter ignores the cut-alternatives in this case. The evolution of the stack in fig. 8 mirrors the example in section 2.1: we evaluate the first goal of the first alternative. During backchaining, we add a pointer to the remaining alternatives to each new cut operator.

If a cut is evaluated, we replace the remaining alternatives with the cut-to alternatives. For example, in fig. 8f, evaluating the cut discards the second and third elements in the stack. Backtracking is performed by simply consuming the first alternative from the stack, as shown in fig. 8d.

4.2 Inadequacy of the stack-based semantics for formal reasoning

The lemmas that we presented in section 3.4 are hard to express in the stack-based semantics, due to the lack of structure in the stack to delimit the scope of the cut operator. For example, consider a stack $a_0, a_1, \dots, a_n$. If a_0 succeeds but executes a cut, it is hard to relate the remaining alternatives $a_1, \dots, a_n$ to the alternatives $b_1, \dots, b_m$ that survive the cut, since the only simple invariant is that $b_1, \dots, b_m$ is a suffix of $a_1, \dots, a_n$. If $a_1, \dots, a_n$ were generated before the call whose execution generates the cut in a_0 , then they all stay; but if some were generated afterwards, they are discarded. Expressing this precisely would require attaching, at the very least, time-stamps (or depths) to goals, which amounts to a cumbersome encoding of a tree.

5 Equivalence of the two semantics

In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree and the stack of alternatives. We define a translation from trees to stacks, called `t2l`, but we do not explicitly provide the converse translation, an economy allowed by the proof technique we employ, inspired by [2].

Before describing the translation of trees to stacks (in section 5.2) we need to define the notion of *valid tree*. The `tree` datatype indeed contains trees that cannot be generated by `runT` via `step` or `prune`, for example all the trees that do not correspond to a depth-first left-to-right tree construction strategy.

5.1 Valid trees

The class of valid trees is captured by the `valid_tree` function shown in Figure 9. While all base cases of `tree` are valid, we need to analyze the `Or` and `And` constructors more carefully.

For the `Or` constructor, we distinguish two cases depending on whether the left-hand side is present. If it is not, then the right-hand side must be a valid tree. Otherwise, the left-hand side must be a valid tree, and the right-hand side must either be the `KO` tree or be unexplored. The former case occurs when the right-hand side is cut away by the evaluation of a superficial cut in the left-hand side. In the latter case we use the `base_or` predicate to check that `B` is the right-skewed tree formed by a disjunction of conjunctions, generated after a successful backchain for a `call`.

For the `And` constructor, the left hand side is required to be a valid tree. The shape of the right hand side depends on whether the left hand side is a success. If it is not successful, then the right hand side must not be explored, i.e. `B` must be the right-skewed tree containing conjunctions of premise atoms. This is enforced using the `big_and` procedure, which generates a tree from the reset point B_0 (the same procedure used to transform each alternative output by `backchain` into the And-layer of the resulting tree). When the left hand side is successful, then the right hand side may have been explored, and consequently must be a valid tree.

```

Fixpoint valid_tree A :=
  match A with
  | Todo _ | OK | KO => T
  | Or □ _ B => valid_tree B
  | Or ·A _ B => valid_tree A &&
    ((B == KO) || B.base_or B)
  | And A B0 B => valid_tree A &&
    if success A then valid_tree B ...
    else B == big_and B0
  end.

Fixpoint t2l t σ (cp: alts) : alts :=
  match t with
  | OK => [(σ, ∅)]
  | KO => ∅
  | TA a => [(σ, [(a, ∅)])]

```

Fig. 9: Valid tree and Tree to stack

TODO: explain better valid tree as requested by reviewer, maybe make a few examples

t2l should be t2s or
tree_to_stack

5.2 From trees to stacks

The **t2l** recursive function translates a tree to a stack. For space constraints fig. 9 only gives the base cases, while we describe the recursive cases in the following text.

The function **t2l** takes as input the tree t_0 , a substitution σ_0 , and a list of choice points cp . These choice points represent alternatives that appear at the same level of the current tree, such as, for example, the right siblings of an **Or** node. The substitution is used to determine under which substitution the subtree being computed lives.

The **OK** node represents one successful alternative; therefore it is translated into a singleton list whose only element has an empty list of goals. The **KO** node represents failure, hence it is mapped to the empty list of alternatives. Finally, atoms are mapped to a singleton list containing one alternative whose only goal is that atom, with an empty cut-to list. At this stage, the cut-to list cannot point to cp : atoms are not right-uncle subtrees, whereas cp represents alternatives that will actually be discarded if the atom turns out to be a cut. The correct cut-to list is therefore determined later, once the trees corresponding to sibling subtrees have been inspected.

The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B . The alternatives of B are valid choice points for A and should be passed to the translation of A . The two lists of alternatives can then be concatenated. Moreover, every atom occurring one level below cp (i.e. in A or B) should add cp as its cut-to alternatives.

The translation of $A \wedge_{B_0} B$ starts with the translation of A . If the translation of A is an empty list we return the empty list of alternatives without even touching B nor B_0 , since an empty translation for A indicates failure and this in turn implies the failure of the entire conjunction. When the translation of A is a list $(\sigma_0, g) :: a$, the B node represents the continuation of the first alternative g , whereas B_0 is the continuation for each alternative in a .

Example of t2l execution The translations of the trees in fig. 5 are shown in fig. 8. The letters in the figures relate each tree to the corresponding stack. For example, 5b is translated into 8b. We also note that 5d.1 and 5d.2 are both translated into 8d, showing that *t2l* is not injective: different trees can map to the same stack. Consequently, there are transitions in **runT** that are not visible in **runS**. As an example of a call to **t2l**, we take the tree and the stack in fig. 10, which are respectively taken from figs. 5c and 8c. The red arrows point from the head of an alternative in the tree to the head

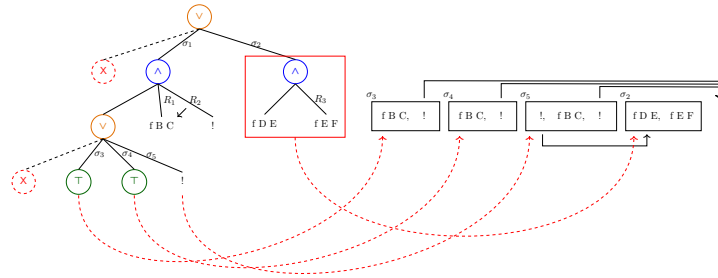


Fig. 10: Example of **t2l** execution

of the corresponding cell in the stack. We focus on the deepest **Or** node in the tree. This subtree is a disjunction with four children. The first is ignored, since it is $\square$. The success node below σ_3 has **f B C** and the cut operator as continuation, corresponding to the first cell in the stack. We note that the cut operator points outside the stack, since the scope of this cut eliminates its right uncle. The success node below σ_4 has R_1 as continuation, where R_1 is the list of goals **f B C, !**; this is translated into the second cell of the stack. Finally, the cut operator under σ_5 also has R_1 as continuation. It corresponds to the third alternative. We can see that the first cut points to the translation of the subtree under σ_2 , since it is one **Or**-layer above its right uncles.

5.3 Equivalence proof

The tree-based semantics exposes more information than needed, hence we hide it behind existentials.

Definition 1. $\text{runT}' \mathcal{P} v \sigma t r := \exists b v', \text{runT} \mathcal{P} v \sigma t r b v'$

From figs. 5 and 8, we observe that, upon backtracking, **runT** may perform more steps than **runS** before returning a solution or a failure. These silent transitions must be skipped when proving the equivalence between the two semantics. Therefore, we prove the following lemma.

Lemma 12 (pruneF.t2l). $\forall t t' b, \text{valid_tree } t \rightarrow \text{failed } t \rightarrow \text{prune } b t = .t' \rightarrow$
 $\forall l \sigma, \text{t2l } t \sigma l = \text{t2l } t' \sigma l.$

Proof. By induction on the tree t .

The first direction of the equivalence states that the execution of a valid tree is matched by the execution of the stack obtained by translating the tree. Moreover, the unexplored alternatives returned as a tree correspond to those returned as a stack.

rename **runT** to
runS

Theorem 1 (tree.to_elpi). $\forall p t \sigma r, \text{let } v := \text{vars_tree } t \cup \text{vars_sigma } \sigma \text{ in}$

$$\begin{aligned} &\text{valid_tree } t \rightarrow \\ &\text{runT}' p v \sigma t r \rightarrow \\ &\quad \text{let } a := \text{t2l } t \sigma \emptyset \text{ in} \\ &\quad \text{let } r' := \text{if } r \text{ is } .(\sigma', t) \text{ then } .(\sigma', \text{t2l } (\text{odflt } KO t) \sigma \emptyset) \\ &\quad \quad \text{else } \square \text{ in} \\ &\text{runS } p v a r'. \end{aligned}$$

Proof. We proceed by induction on the execution of **runT'**. There are four cases to consider. The first case arises from *StopT*; it deals with successful trees. A successful tree corresponds to a stack that start with an empty list, and therefore we conclude using *StopS*. The case for *StepT* requires treating two subcases: **step** evaluates either a cut or a call. Accordingly, we apply *CutS* or *CallS*, followed by the induction hypothesis. The case for *BackT* is silent: it represents the non-injective behavior of **t2l**; however, thanks to lemma 12 and the induction hypothesis, the conclusion is proved. The last case arises from the derivation *FailT*. It is easily proved using *FailS* and the fact that if **prune** returns $\square$ when trying to erase failure in t , then **t2l** applied to t returns the empty list.

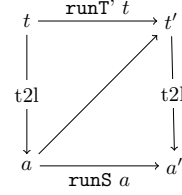
rename runS to
runT

The converse direction is stated following [2]: instead of using a function to translate a stack into a tree it states that any tree that translates to the given stack has an equivalent execution, i.e. gives the same solution and returns an unexplored tree that translates to the unexplored stack.

Theorem 2 (elpi_to_tree). $\forall p \ v \ a \ r, \text{runS } p \ v \ a \ r \rightarrow$

$\forall \sigma \ t, \text{valid_tree } t \rightarrow \text{t2l } t \ \sigma \ \emptyset = a \rightarrow$
if $r \text{ is } (\sigma', a') \text{ then}$
 $\exists t', \text{runT}' \ p \ v \ \sigma \ t \cdot (\sigma', t') \wedge \text{t2l } (\text{odflt } KO \ t') \ \sigma \ \emptyset = a'$
else $\text{runT}' \ p \ v \ \sigma \ t \ \square.$

Proof. The proof is based on the idea explained in [2, Section 3.5.1] and depicted in the figure on the right: we have a tree t whose translation is a , we proceed by induction on the execution of **runS**. This gives us not only the output a' but that hypothesis that it exists a tree t' such that its translation to a stack is a' .



Stating the converse direction in a symmetric way would require a function **12t** transforming a stack a into a tree t . Writing this function is far from trivial, since the structure of the tree is lost by **save_as** and **runT** that intuitively keep the state as a big disjunction of conjunctions by distributing conjunctions over disjunctions. Moreover, one would need to define a notion of valid stack that is equally intricate.

The constructive nature of the logic of Rocq tells us that the proof above provides an effective function translating stacks into trees. However, that function also uses an execution trace of the stack-based semantics, from which it recovers the missing tree structure.

6 Case study: determinacy analysis

An interesting application of the tree semantics is determinacy checking, introduced in version 3 of the ELPI language [12]. The checker takes as input the signatures of the predicates declared by the user and verifies that the rules of a deterministic predicate generate at most one solution; we call this particular kind of predicate a function. Thanks to this static verification, the user can better control unexpected backtracking.

A predicate behaves deterministically if two conditions are satisfied: *mutual exclusion* guarantees that at most one rule can be successfully applied to any given query, whereas *rule checking* ensures that each rule does not create choice points, for example by calling non-deterministic predicates.

Determinacy checking is strongly related to the modes of predicates: we differentiate between input and output arguments. A deterministic call relates to its inputs only one output. In the literature, we find several works on determinacy checking, namely [25,17,8]. These works need a mode checker that statically ensures that, in a query with ground inputs, each recursive call is performed with ground inputs.

Our mechanization implements the unification algorithm by Martelli and Montanari [20] that we prove terminating (using the standard lexicographic ordering), correct and complete.

Definition 2 (acyclic). $\text{acyclic } s := \text{dom } s \cap \text{codom } s = \emptyset$

Note that this definition of acyclicity entails that the substitution is necessarily self applied, i.e. $\sigma := \{X \mapsto f Y, Y \mapsto a\}$ is not acyclic, while the equivalent $\sigma' := \{X \mapsto f a\}$ is. The algorithm by Martelli and Montanari generates substitutions that are acyclic in this sense.

Lemma 13 (Unif.correct).

$$\forall t_1 t_2 \sigma \sigma', \text{acyclic } \sigma \rightarrow \text{unify } t_1 t_2 \sigma = \sigma' \rightarrow \sigma' t_1 = \sigma' t_2$$

Lemma 14 (unif.complete).

$$\forall t_1 t_2 \sigma, \text{acyclic } \sigma \rightarrow (\exists \sigma', \text{acyclic } \sigma' \wedge \sigma' (\sigma t_1) = \sigma' (\sigma t_2)) \rightarrow \exists \sigma'', \text{unify } t_1 t_2 \sigma = \sigma''$$

The ELPI language adopts a restricted unification routine for input arguments, called **matching** [1,30]. Similarly to **unify**, **matching** takes the query term t_1 and the pattern t_2 and returns an optional substitution σ' that does not assign any variable in the query arguments that are in input position.

In order to address this need we extend the Martelli and Montanari unification procedure to take a set of frozen variables, i.e. variables that cannot be assigned. The function **matching** has the following property:

Lemma 15 (matching.disj).

$$\forall t_1 t_2 \sigma \sigma', \text{acyclic } \sigma \rightarrow \text{matching } t_1 t_2 \sigma = \sigma' \rightarrow \sigma' (\sigma t_1) = \sigma t_2$$

The statement above does not say that $\sigma' t_2 = t_2$. Did you prove it?

In particular, we can see how the returned substitution σ' , differently from lemma 13, does not affect variables in σt_2 .

The **backchain** function uses **matching** or **unify** on the arguments q depending if the argument is in *input* or *output* mode.

For the static checker, and in particular in order to validate rule mutual exclusion, we need to ensure that no two rules can be used on the same query in the absence of cut. We exploit the behavior of the **matching** over input argument to guarantee this property: mutual exclusion ensure that every pair of rules for a deterministic predicate have an input term that does not satisfy unification.

An important property we derive from the **matchin** and **unify** procedure is that given three variable-disjoint terms h_1 , h_2 and q , representing respectively two rule heads in the database and a query, if both heads are matching the query q then the two heads must unify. This idea is formalized by the following lemma.

Lemma 16 (unify_matching_trans).

$$\forall h_1 h_2 q, \text{disjoint } h_1 h_2 \rightarrow \text{disjoint } h_1 q \rightarrow \text{disjoint } h_2 q \rightarrow \\ \text{matching } h_1 q \varepsilon \rightarrow \text{matching } h_2 q \varepsilon \rightarrow \text{unify } h_1 h_2 \varepsilon$$

Proof. By lemma 15, we now that the matching of h_1 (resp. h_2) and q gives a substitution σ_1 (resp. σ_2) where the variables in q are not assigned, i.e. its domain only contains variables in h_1 (resp h_2). Using lemma 14, on the conclusion, we need to prove that it exists a substitution σ' such that $\sigma' h_1 = \sigma' h_2$. This substitution is the composition of σ_1 and σ_2 , that is $\sigma' = \lambda x. \sigma_1(\sigma_2 x)$. Since h_1 and h_2 are disjoint, also are the domain of σ_1 and σ_2 , therefore $\sigma' h_1 = \sigma_2(\sigma_1 h_1)$ and $\sigma' h_2 = \sigma_2(\sigma_1 h_2)$. We can now conclude the proof: $\sigma' h_1 = \sigma_2(\sigma_1 h_1) = \sigma_2 q = q = \sigma_2 h_2 = \sigma' h_2$.

Finally, we also need a lemma certifying that the refreshing variables operation performed at runtime during backchaining does not affect unification. In particular, a refreshing renaming f for a term t is a function from variables to variables that: (i) f is defined on all the variables in t , i.e. all variables get renamed; (ii) f is injective, i.e. does not confuse variables; and (iii) f has non-overlapping domain and codomain, i.e. it maps the variables of the term to a set of completely fresh variables.

Below, we define **refresh_for** to test the validity of a refreshing renaming. Lemma 17 states the property of term unifiability with respect to term renaming. That is, if the renamings of terms t_1 and t_2 are unifiable under two respective valid refreshing renamings, and they have disjoint variables, then for any pair of renamings z and x that are valid renamings for t_1 and t_2 , respectively, and are disjoint from each other, unification also succeeds. In other words, this theorem accounts for α -equivalence of terms and the unification of terms with disjoint variables.

Definition 3 (**refresh_for** x t). $(\text{vars } t \subseteq \text{domf } x) \wedge \text{injective } x \wedge (\text{domf } x \# \text{codomf } x)$.

Lemma 17 (**unif_ren**). $\forall x \ y \ z \ w \ t_1 \ t_2,$

$$\begin{aligned} & \text{refresh_for } w \ t_1 \rightarrow \text{refresh_for } y \ t_2 \rightarrow \text{refresh_for } z \ t_1 \rightarrow \text{refresh_for } x \ t_2 \rightarrow \\ & \text{codomf } w \# \text{vars } (\text{rename } y \ t_2) \rightarrow \text{codomf } z \# \text{vars } (\text{rename } x \ t_2) \rightarrow \\ & \text{unify } (\text{rename } w \ t_1) \ (\text{rename } y \ t_2) \ \varepsilon \rightarrow \text{unify } (\text{rename } z \ t_1) \ (\text{rename } x \ t_2) \ \varepsilon. \end{aligned}$$

In fig. 1, the rules of the program are equipped with three signatures, one per predicate. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, their types, and whether the predicate is deterministic. They indicate that **f** and **g** are expected to be functions, as specified by the **func** keyword, whereas **r** is a relation, as indicated by the **pred** keyword. The **->** symbol separates inputs from outputs; therefore, all three predicates are expected to take an **int** as input and return an **int** as output.

The signatures of the program are stored in the **sig** field of the program and are encoded as described in [12]: we distinguish data from predicates, and predicates are classified into deterministic a non-deterministic.

A program execution behaves deterministically if, given a program, a substitution, and an input tree for the **runT** inductive, the execution either fails or, if it succeeds, the resulting tree of alternatives is $\square$.

Definition 4 (**is_det** p σ v t). $\forall r, \text{runT}' \ p \ v \ \sigma \ t \ r \rightarrow r = \square \vee \exists \sigma, r = \cdot(\sigma, \square)$.

The theorem we want to prove is the following one:

Theorem 3 (**det_check_call**). $\forall p \ \sigma \ t \ v,$

$$\text{check}_{\mathbb{P}} \ p \rightarrow \text{tm_is_det } p.(\text{sig}) \ t \rightarrow \text{is_det } p \ \sigma \ v \ (\text{Todo } (\text{call } t)).$$

Here, **check_P** denotes the function that checks the program with respect to mutual exclusion and rule checking, and **tm_is_det** denotes the function that tests whether the head of the term refers to a rigid constant with a deterministic type.

The proof of this lemma proceeds by induction on the program execution. However, without further work, the induction hypothesis is too weak since it talks about **Todo** trees (i.e. atoms), whereas we need it on any tree.

To address this issue, we introduce the notion of deterministic trees (**det_tree**), show that a tree satisfying this condition enjoys the desired properties, and then prove the following more general theorem:

Lemma 18 (**det_check_tree**). $\forall \sigma \ v \ p \ t, \text{check}_{\mathbb{P}} \ p \rightarrow \text{det_tree } p.(\text{sig}) \ t \rightarrow \text{is_det } p \ \sigma \ v \ t.$

Our target theorem **det_check_call** is a corollary of **det_check_tree**.

7 Related work

We studied a PROLOG semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the ELPI interpreter, which is in turn inspired by B-Prolog’s “matching-clauses” [1,30]. The two semantics we presented (and proved equivalent) also apply to standard PROLOG if one forces all predicate signatures to have only output arguments.

For determinacy analysis, matching input arguments affects the proofs only as follows: lemma 15 does require the query q to be ground. Adapting our results to standard PROLOG therefore requires the insertion of two additional hypotheses in theorem 3. First, one has to require that t has ground inputs. Second, that the program p is well-moded. Well-moded predicates produce ground outputs when given ground inputs; hence, starting from an initial query with ground inputs, well-moded programs behave exactly as if input arguments were matched. Therefore, the proofs in section 6 still apply.

The only mechanization of Prolog’s semantics we could find is the one by Pusch in ISABELLE/HOL [24] that is based on the model of Debray and Mishra [7, Sec. 4.3]. Their work starts from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [29,1]. They do not use the semantics to prove properties on the execution of programs as we do in our case study, hence they do not face the difficulty described in section 4 that pushed us to develop a more explicit semantics (with respect to cut). In a sense, our work is complementary. By combining both works one can show the tree-based semantics is equivalent to an optimized stack-based one.

Another relevant work is by Kriener and King [16], but we could not find their Rocq source code. Their semantics is denotational and cannot easily cover all the features that [12] covers, but it would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by [2], where Bertot mechanizes the correctness of a compiler. He relates the semantics of an imperative language to that of its compiled assembly code and proves their equivalence. His approach uses the compiler as the translation function from one language to the other, but does not introduce a decompiler, just as we do not define a function from stacks to trees. To show that the assembly code behaves like the imperative language, he proceeds by induction on the execution of the assembly program. We adopt the same strategy in the proof of theorem 2.

8 Conclusion

This paper describes two operational semantics for Prolog with cut. The first semantics is based on And-Or trees: it is well suited both to graphical illustrations of the scope of cut and to proving lemmas about the effect of cut evaluation when reasoning about disjunctions. The second semantics is based on a stack of alternatives: it is closer to actual PROLOG implementations, including the first-order fragment of ELPI, but it loses the explicit structure of the tree-based semantics and is therefore less convenient for reasoning about cut. We show how to translate trees into stacks and prove the two semantics equivalent. Using the tree-based semantics, we mechanize a soundness proof of the first-order fragment of the determinacy checker of [12], namely that a call to a deterministic predicate returns at most one solution.

The mechanization consists of approximately 2,500 lines of specifications and about 5,000 lines of ssreflect proofs based on the Mathematical Components library [19] and its finmap [21] extension. Roughly 30% of the code is dedicated to the tree semantics, in particular the proofs in section 3.4, 15% to the stack semantics, and 35% to the equivalence proof. The remaining 20% is devoted to the determinacy checker. The mechanization took us approximately 8 months. We believe that the ssreflect proof language, and the proof style it advocates, helped us carry out the many refactorings that the mechanization underwent. The finmap library provides comprehensive support for reasoning about finite maps and finite sets over a countable domain. In particular, it helped us express conditions such as the disjointness between sets of variables and the domain of a substitution, as well as define the union of disjoint substitutions.

To obtain an axiom-free mechanization, we had to provide an implementation of the first-order unification algorithm, since we could not find an existing Rocq mechanization. Describing this development in detail is out of scope for this paper. It amounts to XXX lines of code, including YY lines for the termination proof based on a lexicographic ordering and ZZ lines for correctness and completeness. Because of the specific form of Lemma XXX, which deals with matching terms with disjoint support, we did not need to prove that the algorithm produces most-general unifiers.

To fully model the ELPI language and mechanize [12] entirely, we would need to account for higher-order variables, i.e. variables representing predicates. This extension is ongoing and largely orthogonal to the present paper, since cut and higher-order variables do not interact in subtle ways. The main difficulty is that substitutions may contain predicates, and these predicates must have a signature compatible with the one assumed statically, in the sense of the $\subseteq$ relation defined in [12, Section 5].

Acknowledgments. We thank Yves Bertot and Laurent Theiry for insightful discussions on how to prove the equivalence of the two semantics.

Ce travail a bénéficié d’une aide du gouvernement français, gérée par l’Agence Nationale de la Recherche au titre du Plan d’investissement France 2030, dans le cadre du projet ”UCA DS4H” portant la référence n° ANR-17-EURE-0004.

References

1. Aït-Kaci, H.: Warren’s Abstract Machine: A Tutorial Reconstruction. The MIT Press (08 1991). <https://doi.org/10.7551/mitpress/7160.001.0001>, <https://doi.org/10.7551/mitpress/7160.001.0001>
2. Bertot, Y.: A certified compiler for an imperative language. Tech. Rep. RR-3488, INRIA (September 1998), <https://inria.hal.science/inria-00073199v1>
3. Blot, V., Cousineau, D., Crance, E., de Prisque, L.D., Keller, C., Mahboubi, A., Vial, P.: Compositional pre-processing for automated reasoning in dependent type theory. In: Krebbers, R., Traytel, D., Pientka, B., Zdanczewicz, S. (eds.) Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023. pp. 63–77. ACM (2023). <https://doi.org/10.1145/3573105.3575676>, <https://doi.org/10.1145/3573105.3575676>
4. Cohen, C., Crance, E., Mahboubi, A.: Trocq: Proof transfer for free, with or without univalence. In: Weirich, S. (ed.) Programming Languages and Systems - 33rd

Martelli (size, time, relation with saarland (maybe don’t say), substitutions are not finite maps, hence badly integrate with math-comp, also the algorithm is not explicitly given)

- European Symposium on Programming, ESOP 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6-11, 2024, Proceedings, Part I. pp. 239–268. Lecture Notes in Computer Science, Springer (2024). https://doi.org/10.1007/978-3-031-57262-3_10, https://doi.org/10.1007/978-3-031-57262-3_10
5. Cohen, C., Crance, E., Mahboubi, A.: TrocQ: Proof transfer for free, beyond equivalence and univalence. *ACM Trans. Program. Lang. Syst.* **47**(3), 12:1–12:40 (2025). <https://doi.org/10.1145/3737283>, <https://doi.org/10.1145/3737283>
 6. Cohen, C., Sakaguchi, K., Tassi, E.: Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In: *Proceedings of FSCD. LIPIcs*, vol. 167, pp. 34:1–34:21 (2020). <https://doi.org/10.4230/LIPIcs.FSCD.2020.34>, <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34>
 7. Debray, S.K., Mishra, P.: Denotational and operational semantics for prolog. *J. Log. Program.* **5**(1), 61–91 (Mar 1988). <https://doi.org/Debray1988>, <https://doi.org/Debray1988>
 8. Debray, S.K., Warren, D.S.: Functional computations in logic programs. vol. 11, p. 451–481. *Association for Computing Machinery* (Jul 1989). <https://doi.org/10.1145/65979.65984>, <https://doi.org/10.1145/65979.65984>
 9. Dunchev, C., Guidi, F., Coen, C.S., Tassi, E.: ELPI: fast, embeddable, λ Prolog interpreter. In: *Proceedings of LPAR. LNCS*, vol. 9450, pp. 460–468. Springer (2015). https://doi.org/10.1007/978-3-662-48899-7_32, <https://inria.hal.science/hal-01176856v1>
 10. Fissore, D., Tassi, E.: A new Type-Class solver for Coq in Elpi. In: *The Coq Workshop* (Jul 2023), <https://inria.hal.science/hal-04467855>
 11. Fissore, D., Tassi, E.: Higher-order unification for free!: Reusing the meta-language unification for the object language. In: *Proceedings of PPDP*. pp. 1–13. ACM (2024). <https://doi.org/10.1145/3678232.3678233>, <https://doi.org/10.1145/3678232.3678233>
 12. Fissore, D., Tassi, E.: Determinacy checking for elpi: an higher-order logic programming language with cut. In: Amin, N., Arias, J. (eds.) *Practical Aspects of Declarative Languages*. pp. 77–95. Springer Nature Switzerland, Cham (2026)
 13. Grégoire, B., Léchenet, J.C., Tassi, E.: Practical and sound equality tests, automatically. In: *Proceedings of CPP*. p. 167–181. *Association for Computing Machinery* (2023). <https://doi.org/10.1145/3573105.3575683>, <https://doi.org/10.1145/3573105.3575683>
 14. Guidi, F., Coen, C.S., Tassi, E.: Implementing type theory in higher order constraint logic programming. In: *Mathematical Structures in Computer Science*. vol. 29, pp. 1125–1150. Cambridge University Press (2019). <https://doi.org/10.1017/S0960129518000427>
 15. Krebbers, R., van der Maas, L., Tassi, E.: Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In: Forster, Y., Keller, C. (eds.) *16th International Conference on Interactive Theorem Proving (ITP 2025)*. *Leibniz International Proceedings in Informatics (LIPIcs)*, vol. 352, pp. 27:1–27:21. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2025). <https://doi.org/10.4230/LIPIcs.ITP.2025.27>, <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27>
 16. Kriener, J., King, A.: Redalert: Determinacy inference for prolog. vol. 11, pp. 537–553. Cambridge University Press (2011)
 17. Lu, L., King, A.: Determinacy inference for logic programs. vol. 3444, pp. 108–123 (04 2005). https://doi.org/10.1007/978-3-540-31987-0_9

18. van der Maas, L.: Extending the Iris Proof Mode with Inductive Predicates using Elpi. Master's thesis, Radboud University Nijmegen (2024). <https://doi.org/10.5281/zenodo.12568604>
19. Mahboubi, A., Tassi, E.: Mathematical Components. Zenodo (Sep 2022). <https://doi.org/10.5281/zenodo.7118596>, <https://doi.org/10.5281/zenodo.7118596>
20. Martelli, A., Montanari, U.: An efficient unification algorithm. *ACM Trans. Program. Lang. Syst.* **4**(2), 258–282 (Apr 1982). <https://doi.org/10.1145/357162.357169>, <https://doi.org/10.1145/357162.357169>
21. math-comp: finmap — finite maps for mathcomp (2026), <https://github.com/math-comp/finmap>
22. Miller, D.: A logic programming language with lambda-abstraction, function variables, and simple unification. In: *Extensions of Logic Programming*. pp. 253–281. Springer (1991)
23. Miller, D., Nadathur, G.: *Programming with Higher-Order Logic*. Cambridge University Press (2012)
24. Pusch, C.: Verification of compiler correctness for the wam. In: Goos, G., Hartmannis, J., van Leeuwen, J., von Wright, J., Grundy, J., Harrison, J. (eds.) *Theorem Proving in Higher Order Logics*. pp. 347–361. Springer Berlin Heidelberg, Berlin, Heidelberg (1996)
25. Somogyi, Z., Henderson, F., Conway, T.: The execution algorithm of mercury, an efficient purely declarative logic programming language. vol. 29, pp. 17–64 (1996). [https://doi.org/https://doi.org/10.1016/S0743-1066\(96\)00068-4](https://doi.org/https://doi.org/10.1016/S0743-1066(96)00068-4), <https://www.sciencedirect.com/science/inproceedings/pii/S0743106696000684>, high-Performance Implementations of Logic Programming Systems
26. SWI-Prolog Documentation: Predicate `!/0` — cut (built-in predicate) (2026), https://www.swi-prolog.org/pldoc/doc_for?object=!/0, accessed via SWI-Prolog PiDoc reference
27. Tassi, E.: Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λ Prolog dialect). In: *The Fourth International Workshop on Coq for Programming Languages* (Jan 2018), <https://inria.hal.science/hal-01637063>
28. Tassi, E.: Deriving proved equality tests in Coq-Elpi. In: *Proceedings of ITP. LIPIcs*, vol. 141, pp. 29:1–29:18 (Sep 2019). <https://doi.org/10.4230/LIPIcs.CVIT.2016.23>, <https://inria.hal.science/hal-01897468>
29. Warren, D.H.: An Abstract Prolog Instruction Set. Tech. Rep. Technical Note 309, SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA (October 1983), <https://www.sri.com/wp-content/uploads/2021/12/641.pdf>
30. Zhou, N.f.: The language features and architecture of b-prolog. *Theory Pract. Log. Program.* **12**(1–2), 189–218 (Jan 2012). <https://doi.org/10.1017/S1471068411000445>, <https://doi.org/10.1017/S1471068411000445>